Teoria Współbieżności

Kajetan Frątczak 03.01.2026 r.

Laboratorium 10

Teoria Śladów cz.2

Zadanie

Napisz program w dowolnym języku, który:

1. Wyznacza relację niezależności I (poprawka - ma wyznaczać D - relację zależności, czyli dopełnienie I)
2. Wyznacza ślad [w] względem relacji I
3. Wyznacza postać normalną Foaty FNF([w]) śladu [w]
4. Wyznacza graf zależności dla słowa w
5. Wyznacza postać normalną Foaty na podstawie grafu

Dane są:

- Alfabet A, w którym każda litera oznacza akcję.
- Relacja niezależności I, oznaczająca które akcje są niezależne (przemienne, tzn. można je wykonać w dowolnej kolejności i nie zmienia to wyniku końcowego).
- Słowo w oznaczające przykładowe wykonanie sekwencji akcji.

Uwagi:

- Proszę wykorzystać algorytm ze str. 10 rozdziału Partial commutation and traces (pochodzi z Handbook of Formal Languages, Springer, 1997.
- Do rysowania grafu można wykorzystać Graphviz i format DOT. Wersja online: Webgraphviz.

Program w języku python:

```
# Trace Theory

# Program ma za zadanie:
# Wyznaczać relację niezależności I (poprawka - ma wyznaczać D - relację zależności, czyli
dopełnienie I)
# Wyznaczać ślad [w] względem relacji I
# Wyznaczać postać normalną Foaty FNF([w]) śladu [w]
# Wyznaczać graf zależności dla słowa w
# Wyznaczać postać normalną Foaty na podstawie grafu

# Dane testowe nr 1
# A = {'a', 'b', 'c', 'd'}
# I = {('a', 'd'), ('d', 'a'), ('b', 'c'), ('c', 'b')}
# w = "baadcb"
```

```python
# Dane testowe nr 2
A = {'a', 'b', 'c', 'd', 'e', 'f'}
I = {('a', 'd'), ('d', 'a'), ('b', 'e'), ('e', 'b'), ('c', 'd'), ('d', 'c'), ('c', 'f'), ('f', 'c')}
w = "acdcfbbe"

# 1. Wyznaczanie D na podstawie alfabetu oraz relacji niezależności
# D - relacja ta określa które pary akcji nie mogą być zamieniane kolejnością
D = set()
for x in A:
    for y in A:
        # jeśli para nie jest w I, to jest w D
        if (x,y) not in I:
            D.add((x,y))
D = sorted(list(D))

print("D: ", D)
print("\n")

# 2. Wyznaczanie śladu [w] względem relacji niezależności I
# Możemy zamieniać kolejnościa w 'w' jedynie pary symboli z I
trace = {w}

queue = [w]

while(queue):
    current_word = queue.pop(0)
    for i in range(len(current_word)-1):
        x,y = current_word[i], current_word[i+1]
        if (x,y) in I or (y,x) in I:
            #print(i, i+1)
            new_word = current_word[:i] + y + x + current_word[i+2:]
            if new_word not in trace:
                  trace.add(new_word)
                  queue.append(new_word)
print("Ślad: ", trace)
print("\n")

# 3. Wyznaczanie postaci normalnej Foaty FNF([w]) śladu [w]

# Najpierw zgodnie z algorytmem opisanym w artykule tworzymy stosy dla każdej litery alfabetu
stacks = {letter: [] for letter in A}

# Przechodzimy przez słowo od prawej do lewej
for char in reversed(w):
    if char not in A:
        print("Error: litera spoza alfabetu.")
    stacks[char].append(char)
    for other_char in A:
        if other_char != char and ((char, other_char) in D or (other_char, char) in D):
            stacks[other_char].append('*')

#print(stacks)

# Zdejmowanie ze stosów i budowa FNF
fnf = []

while True:
    # kandydat - litera na szczycie swojego stosu
    curr_step = []
    all_empty = True
```

```
    for char in A:
        if stacks[char]:
            all_empty = False
            top = stacks[char][-1]
            if top != '*':
                curr_step.append(top)

    if all_empty:
        break

    # Sortujemy litery w kroku
    curr_step.sort()
    fnf.append("".join(curr_step))

    # Zdejmujemy wybrane litery i odpowiadające im znaczniki
    for char_to_pop in curr_step:
        stacks[char_to_pop].pop()

        # Zdejmuje znaczniki '*' ze stosów liter zależnych
        for other_char in A:
            if other_char != char_to_pop and ((char_to_pop, other_char) in D or (other_char,
char_to_pop) in D):
                stacks[other_char].pop()
            else:
                pass
fnf = "(" + ")(".join(fnf) + ")"
print("FNF: ", fnf)
print("\n")

# 4. Budowa grafu zależności dla słowa w
n = len(w)
adj = {i: [] for i in range(n)}

# Krok 1: Budujemy pełny graf zależności
# Łączymy każde i z j (gdzie i < j), jeśli litery są zależne
for i in range(n):
    for j in range(i+1, n):
        if (w[i], w[j]) not in I:
            adj[i].append(j)

# Pomocniczy BFS do sprawdzania odległości
from collections import deque

def can_reach(u, v, graph):
    visited = set([u])
    queue = deque([u])
    while queue:
        curr = queue.popleft()
        if curr == v:
            return True
        for nxt in graph[curr]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append(nxt)
    return False

# Krok 2: redukcja (usuwanie nadmiarowych krawędzi)
for u in range(n):
    neighbors_copy = list(adj[u])
```

```python
    for v in neighbors_copy:
        # Tymczasowo usuwamy krawędź u -> v
        adj[u].remove(v)
        # Sprawdzamy, czy nadal da się dojść z u do v
        if not can_reach(u, v, adj):
            # Jeśli się NIE da, to krawędź była niezbędna - przywracamy ją
            adj[u].append(v)
            # Sortujemy, żeby zachować kolejność (opcjonalne, ale ładniej wygląda)
            adj[u].sort()
        # Jeśli can_reach zwróciło True, to znaczy że istnieje inna droga (np. u -> k -> v),
        # więc krawędź bezpośrednia u -> v jest zbędna i pozostaje usunięta.

nodes = []
edges = []

for i in range(n):
    nodes.append((i+1, w[i]))
    for target in adj[i]:
        edges.append((i+1, target+1))

print("Węzły (id, label):", nodes)
print("\n")
print("Krawędzie (src, dst):", edges)
print("\n")

# W formacie dot
dot_code = "digraph g {\n"
for src, dst in edges:
    dot_code += f' {src} -> {dst}\n'
for node_id, label in nodes:
    dot_code += f' {node_id} [label="{label}"]\n'
dot_code += "}"

print("Kod dot: ")
print(dot_code)
print("\n")

# 5. Wyznaczamy postać normalną Foaty na podstawie grafu
# Krok 1 - obliczenie stopni wejściowych (ile krawędzi wchodzi do wierzchołka)
in_degree = {i: 0 for i in range(n)}
for u in range(n):
    for v in adj[u]:
        in_degree[v] += 1

# Krok 2 - warstwy
foata_layers = []
current_layer = [i for i in range(n) if in_degree[i] == 0]

while current_layer:
    # sortowanie indeksów w warstwie według liter, aby było ad a nie da
    current_layer.sort(key=lambda index: w[index])
    # tworzymy napis dla obecnej warstwy
    layer_str = "".join(w[i] for i in current_layer)
    foata_layers.append(layer_str)
    # przygotowujemy następną warstwę
    next_layer = []
    for u in current_layer:
        for v in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
```

```
                next_layer.append(v)
    # przechodzimy do kolejnej warstwy
    current_layer = next_layer

fnf_from_graph = "(" + ")(".join(foata_layers) + ")"
print("FNF wyznaczona z grafu:", fnf_from_graph)
```

Wyniki - dane testowe nr 1:
D: [('a', 'a'), ('a', 'b'), ('a', 'c'), ('b', 'a'), ('b', 'b'), ('b', 'd'), ('c', 'a'), ('c', 'c'), ('c', 'd'), ('d', 'b'), ('d', 'c'), ('d', 'd')]

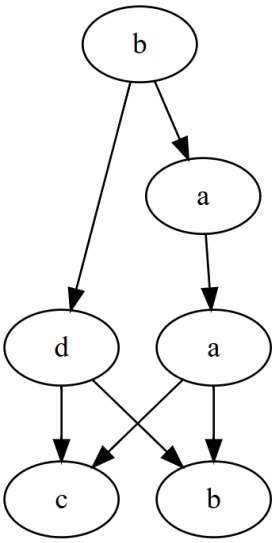
Ślad: {'badacb', 'bdaabc', 'badabc', 'baadcb', 'bdaacb', 'baadbc'}

FNF: (b)(ad)(a)(bc)

Węzły (id, label): [(1, 'b'), (2, 'a'), (3, 'a'), (4, 'd'), (5, 'c'), (6, 'b')]

Krawędzie (src, dst): [(1, 2), (1, 4), (2, 3), (3, 5), (3, 6), (4, 5), (4, 6)]

Kod dot:
```
digraph g {
 1 -> 2
 1 -> 4
 2 -> 3
 3 -> 5
 3 -> 6
 4 -> 5
 4 -> 6
 1 [label="b"]
 2 [label="a"]
 3 [label="a"]
 4 [label="d"]
 5 [label="c"]
 6 [label="b"]
}
```



Rysunek 1: Graf zależności

FNF wyznaczona z grafu: (b)(ad)(a)(bc)

Wyniki - dane testowe nr 2:

D: [('a', 'a'), ('a', 'b'), ('a', 'c'), ('a', 'e'), ('a', 'f'), ('b', 'a'), ('b', 'b'), ('b', 'c'), ('b', 'd'), ('b', 'f'), ('c', 'a'), ('c', 'b'), ('c', 'c'), ('c', 'e'), ('d', 'b'), ('d', 'd'), ('d', 'e'), ('d', 'f'), ('e', 'a'), ('e', 'c'), ('e', 'd'), ('e', 'e'), ('e', 'f'), ('f', 'a'), ('f', 'b'), ('f', 'd'), ('f', 'e'), ('f', 'f')]

Ślad: {'dacfcbbe', 'adccfebb', 'adcfcbeb', 'dafccbeb', 'acdfcebb', 'adccfbbe', 'daccfbbe', 'adfccbbe', 'adcfcebb', 'acdfcbbe', 'dacfcbeb', 'daccfebb', 'adcfcbbe', 'adccfbeb', 'acdcfbbe', 'accdfbbe', 'accdfebb', 'acdcfebb', 'adfccebb', 'acdcfbeb', 'dafccebb', 'dafccbbe', 'acdfcbeb', 'dacfcebb', 'daccfbeb', 'adfccbeb', 'accdfbeb'}

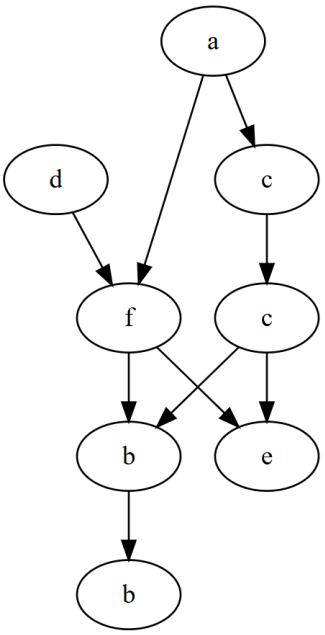
FNF: (ad)(cf)(c)(be)(b)

Węzły (id, label): [(1, 'a'), (2, 'c'), (3, 'd'), (4, 'c'), (5, 'f'), (6, 'b'), (7, 'b'), (8, 'e')]

Krawędzie (src, dst): [(1, 2), (1, 5), (2, 4), (3, 5), (4, 6), (4, 8), (5, 6), (5, 8), (6, 7)]

Kod dot:

```
digraph g {
 1 -> 2
 1 -> 5
 2 -> 4
 3 -> 5
 4 -> 6
 4 -> 8
 5 -> 6
 5 -> 8
 6 -> 7
 1 [label="a"]
 2 [label="c"]
 3 [label="d"]
 4 [label="c"]
 5 [label="f"]
 6 [label="b"]
 7 [label="b"]}
 8 [label="e"]
}
```



Rysunek 2: Graf zależności

FNF wyznaczona z grafu: (ad)(cf)(c)(be)(b)